

CSA3023/CSE3023/CSM3023 WEB-BASED APPLICATION DEVELOPMENT

Lab Modules



WAN NURAL JAWAHIR HJ WAN YUSSOF, RABIEI MAMAT,
MOHD ARIZAL SHAMSIL MAT RIFIN
& MUHAMMAD SYAFFIQ HAZARI

Contents

List of Figures	vi
1 DEVELOPMENT ENVIRONMENT SETUP FOR JAVA WEB APPLICATIONS	1
1.1 Objectives	1
1.2 Introduction	1
1.3 Software Requirements	2
1.4 Task 1: Install Java Development Kit (JDK)	2
1.4.1 Step 1: Download JDK	2
1.4.2 Step 2: Install JDK	2
1.4.3 Step 3: Verify JDK Installation	2
1.5 Task 2: Install XAMPP	3
1.5.1 Step 1: Download XAMPP	3
1.5.2 Step 2: Install XAMPP without Tomcat	4
1.5.3 Step 3: Verify XAMPP Installation	9
1.6 Task 3: Change the Default Root Password of MySQL	11
1.6.1 Step 1: Start Apache and MySQL Services	11
1.6.2 Step 2: Run Command	12
1.7 Task 4: Tomcat 9 Installation	18
1.7.1 Step 1: Download Apache Tomcat 9	18
1.7.2 Step 2: Extract Apache Tomcat 9	19
1.7.3 Step 3: Start Tomcat Server	19
1.7.4 Step 4: Verify Installation	20
1.8 Task 5: NetBeans IDE Installation	21
1.8.1 Step 1: Download Apache NetBeans 29	21
1.8.2 Step 2: Extract and Install NetBeans 29	21
1.8.3 Step 3: Configure NetBeans 29	23
1.8.4 Step 5: Configure Tomcat in NetBeans	23
1.9 Task 6: Writing a Simple Java Servlet	26
1.9.1 Step 1: Create New Project	26
1.9.2 Step 2: Create New Servlet	29
1.9.3 Step 3: Run Servlet	32
1.10 Task 7: Writing a Simple JSP Program	34
1.10.1 Step 1: Create a New Project	34
1.10.2 Step 2: Create a New JSP File	36
1.10.3 Step 3: Run JSP File	39
1.11 Task 8: Use Java Reference Datatype/Class Wrapper in JSP	41
1.11.1 Step 1: Create a New JSP File	41

1.12	Exercise	43
2	SERVLET: DATA SHARING AND DATABASE MANAGEMENT	45
2.1	Objectives	45
2.2	Introduction	46
2.3	Software Requirements	46
2.4	Task 1: Data Sharing in Servlet	46
2.4.1	Step 1: Project Setup	46
2.4.2	Step 2: Create HTML Login Page	48
2.4.3	Step 3: Create a Login Servlet	48
2.4.4	Step 4: Create an Account Servlet	51
2.4.5	Step 5: Run the Project	52
2.5	Task 2: Creating A Table in MySQL Database	53
2.5.1	Step 1: Start Apache and MySQL	53
2.5.2	Step 2: Create a New Database	53
2.5.3	Step 3: Create a New MySQL Table	53
2.6	Task 3: Setting the Environment of Web Application for Database Connection	54
2.6.1	Step 1: Create a New Web Application Project	54
2.6.2	Step 2: Prepare the MySQL Connector	55
2.7	Task 4: Using Servlets for Database CRUD Operations	57
2.7.1	Step 1: Create HTML Form for Data Entry	57
2.7.2	Step 2: Create the Insert Servlet (Create)	58
2.7.3	Step 3: Create the View Servlet (Read)	59
2.7.4	Step 4: Create the Delete Servlet (Delete)	62
2.7.5	Step 5: Create the Update Servlet (Update)	63
2.8	Task 5: Using Servlets, DAO and Java Beans for Database CRUD Operations	65
2.8.1	Step 1: Copy and Rename the Project	66
2.8.2	Step 2: Create the User JavaBean (Model)	66
2.8.3	Step 3: Create the Data Access Object (UserDAO)	68
2.8.4	Step 4: Modify the Existing Servlets (Controller)	72
2.9	Exercise	77
3	JSP: SCRIPTLET, PAGE DIRECTIVE & INCLUDE DIRECTIVE	79
3.1	Objectives	79
3.2	Introduction	79
3.3	Software Requirements	80
3.4	Task 1: Passing Data from Main JSP's Page to Other JSP's Page	80
3.4.1	Step 1: Create A New JSP Page	80
3.4.2	Step 2: Write an HTML Markup	81
3.5	Task 2: Using Mathematics Operations in JSP	84
3.5.1	Step 1: Create A New JSP Page	85
3.5.2	Step 2: Write an HTML Markup	85
3.5.3	Step 3: Run the <code>Calculator.jsp</code>	86
3.6	Task 3: Populate an Array Values into HTML's Table	89
3.6.1	Step 1: Create A New JSP Page	89
3.6.2	Step 2: Write An HTML Markup	89
3.6.3	Step 3: Run the <code>populateArray.jsp</code> File	90

3.7	Task 4: Perform Calculation of Car Loan	90
3.7.1	Step 1: Create A New HTML File	90
3.7.2	Step 2: Create An HTML Markup	91
3.7.3	Step 3: Create <code>processCalculateCarLoan.jsp</code>	92
3.8	Task 5: Using JSP Page Directive to Call Java API	93
3.9	Task 6: Use JSP Include directive for JSP Page	98
4	JSP: SCRIPTLET, EXPRESSION & STANDARD ACTIONS	103
4.1	Objectives	103
4.2	Introduction	103
4.3	Software Requirements	104
4.4	Task 1: Using JSP Scripting	110
4.4.1	Step 1: Create <code>customer.html</code>	110
4.4.2	Step 2: Create <code>processCustomer.jsp</code>	111
4.4.3	Step 3: Compile and Run the Application	113
4.5	Task 2: Using JSP (Scripting, Declaration and Expression)	115
4.5.1	Step 1: Create <code>currencyConversion.html</code>	115
4.5.2	Step 2: Create <code>processCurrency.jsp</code>	116
4.5.3	Step 3: Compile and Run the Application	118
4.6	Task 3: Using JSP Standard Action (Include and Param)	120
4.6.1	Step 1: Create <code>jspParameter.jsp</code>	120
4.6.2	Step 2: Create <code>subjectInfo.jsp</code>	122
4.6.3	Step 3: Compile and Run the Application	123
4.7	Task 4: Using JSP Standard Action (Forward)	124
4.7.1	Step 1: Create <code>forward.jsp</code>	124
4.7.2	Step 2: Create <code>forwardInfo.jsp</code>	125
4.7.3	Step 3: Compile and Run the Application	126
4.8	Task 5: Using JSP Scriptlet to Construct Business Logic	127
4.8.1	Step 1: Create Project and JSP File <code>insuranceQuotation.jsp</code>	127
4.8.2	Step 2: Design Insurance Quotation Interface	127
4.8.3	Step 3: Create <code>processInsuranceQuo.jsp</code>	128
4.8.4	Step 4: Implement Business Logic Using JSP Scriptlet	129
4.8.5	Step 5: Compile and Run the Application	130
5	JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)	135
5.1	Objectives	135
5.2	Introduction	136
5.3	Software Requirements	136
5.4	Task 1: Using Scriptlet to Access a Simple JavaBeans	136
5.4.1	Step 1: Create the JavaBean	136
5.4.2	Step 2: Access Bean via Scriptlet	137
5.5	Task 2: Problem Solving using JavaBeans	138
5.5.1	Step 1: Create a Calculator Bean	138
5.5.2	Step 2: Create Form and Process Page	139
5.6	Task 3: Installing JSTL Taglibs	140
5.6.1	Step 1: Download and Import JSTL Libraries	141
5.6.2	Step 2: Test the Installation and Formatting Tags	142
5.7	Task 4: Using Java Standard Tag Library (JSTL) Core Tags	144

5.7.1	Step 1: Variables and Conditionals	144
5.8	Task 5: Using JSP Standard Tag Library (JSTL) for Collections	145
5.8.1	Step 1: Create the Servlet (Controller)	145
5.8.2	Step 2: Create the View (JSTL forEach)	146
5.9	Task 6: Database Access using JSTL SQL Tags	147
5.9.1	Step 1: Create Database and Table	148
5.9.2	Step 2: Prepare the Database Driver	148
5.9.3	Step 3: Create the JSTL SQL Page	149
5.10	Exercise: Employee Payroll Display System	151
6	JSP: SAVING AND RETRIEVING DATA FROM DATABASE	153
6.1	Objectives	153
6.2	Introduction	153
6.3	Software Requirements	154
6.4	Task 1: Using JSP Page to Access a Simple MySQL Database	155
6.4.1	Step 1 - Create a table namely <code>FirstTable</code> using phpMyAdmin	155
6.4.2	Step 2 - Create <code>SampleInsertionRecord.jsp</code> to insert data in <code>FirstTable</code> table.	156
6.4.3	Step 3 - go to the database.	158
6.5	Task 2: Create Records via JSP Page	158
6.6	Task 3: Create Records Constrained by Regular Expression in JSP	164
6.6.1	Step 1 - Create a table <code>student</code> using phpMyAdmin.	164
6.6.2	Step 2 - Create <code>insertStudent.jsp</code> as a main interface to register a new student.	165
6.6.3	Step 3 - Create <code>Student</code> JavaBeans	165
6.6.4	Step 4 - Create <code>processStudent.jsp</code> to insert record into the database	166
6.6.5	Step 5 - Create <code>errorStudent.jsp</code> to display any error message	168
6.6.6	Step 6 - Running the program and create a new database	169
6.7	Task 4: Perform Retrieving Records Via JSP Page	169
6.8	Task 5: Create A Record Using JSP Model 1	172
7	SESSION MANAGEMENT, FILE UPLOAD & MVC	177
7.1	Objectives	177
7.2	Introduction	177
7.3	Database Preparation	178
7.4	Task 1: Building the Model (JavaBean & DAO)	178
7.4.1	Step 1: Create the <code>StudentBean</code>	179
7.4.2	Step 2: Create the <code>StudentDAO</code>	179
7.5	Task 2: The Unified Controller (UserServlet)	182
7.5.1	Step 1: Create UserServlet with web.xml Mapping	182
7.6	Task 3: The Views and Session Security	184
7.6.1	Step 1: Registration Form	185
7.6.2	Step 2: Login Form	185
7.6.3	Step 3: Secure Dashboard JSP	186
7.7	Task 4: Debugging and Viewing Server Logs	188
7.7.1	Step 1: Simulate a Backend Error	188
7.7.2	Step 2: Inspecting the Apache Tomcat Output Window	188

7.7.3	Step 3: Viewing the Full Server Log via Services Tab	189
7.8	Exercise: Student Subject Management System (CRUD)	191
	APPENDIX: Lecturer Solution Key (Subject CRUD)	193
8	AN MVC EXAMPLE WITH SERVLETS AND JSP	197
8.1	Objectives	197
8.2	Introduction	197
8.3	Software Requirements	198

List of Figures

1.1	XAMPP Download Platform	3
1.2	XAMPP Setup Wizard Startup Screen	5
1.3	XAMPP Select Components	5
1.4	XAMPP Installation Directory	6
1.5	XAMPP Language	7
1.6	XAMPP Installation Ready	7
1.7	XAMPP Installation Progress	8
1.8	XAMPP Installation Finish	8
1.9	XAMPP Control Panel	9
1.10	XAMPP Control Panel Running	10
1.11	XAMPP Dashboard	10
1.12	phpMyAdmin Interface	11
1.13	XAMPP Control Panel-Click Shell	11
1.14	Verify Password	12
1.15	Error Page	13
1.16	Apache Tomcat9 Download Page	18
1.17	Start Tomcat Server	19
1.18	Apache Tomcat Homepage	20
1.19	Apache NetBeans 29 Download Page	21
1.20	Extract netbeans-29-bin.zip	22
1.21	Netbean Executable File	22
1.22	Netbean IDE GUI	23
1.23	Configure Server	24
1.24	Add Server	24
1.25	Add Server	25
1.26	Add Server	25
1.27	Add Server	26
1.28	Create New Project	27
1.29	Create New Web Application	27
1.30	Enter Project Details	28
1.31	Create New Servlet	29
1.32	Name the Servlet	30
1.33	Configure Servlet Deployment	30
1.34	HelloServlet.java File	31
1.35	<code>processRequest()</code> Method	31
1.36	Modified <code>processRequest()</code> Method	31
1.37	Run the Servlet	32

1.38	Dialog Box Call to /HelloServlet	32
1.39	Output	33
1.40	Modified <code>doGet()</code> Method	33
1.41	Dialog Box with Passing Parameter	34
1.42	Output	34
1.43	Create New Web Application	35
1.44	Enter Project Details	36
1.45	Configure the Server	36
1.46	Create a New JSP File	37
1.47	Enter Name and Location	37
1.48	Modified JSP File	38
1.49	Compile JSP File	38
1.50	Compile JSP File	38
1.51	Run JSP File	39
1.52	Output	40
1.53	Start/Stop Apache Tomcat	40
1.54	Create <code>useJavaObject.jsp</code>	41
1.55	<code>useJavaObject.jsp</code>	42
1.56	Output	43
2.1	Create New Servlet	49
2.2	Create Login Servlet	49
2.3	Configure servlet deployment	50
2.4	Creating the “CRUDusingServlet” Project	55
2.5	Downloading MySQL Connector/J	55
2.6	Extracting the MySQL Connector/J Archive	56
2.7	Adding External Library to Project	56
2.8	Importing MySQL Connector/J Library	57
3.1	Create new JSP	81
3.2	New JSP	81
3.3	<code>memberRegister.jsp</code> Interface	82
3.4	Filling input <code>memberRegister.jsp</code>	84
3.5	Output in <code>memberProcessing.jsp</code>	84
3.6	Create new JSP	85
3.7	New JSP	85
3.8	<code>calculator.jsp</code> Interface	86
3.9	Key-in Filename	89
3.10	Output from Array	90
3.11	Creating New HTML File	91
3.12	Interface Output for Car Loan Calculation HTML	91
3.13	Input for Car Loan Calculation	92
3.14	Output for Car Loan Calculation	93
3.15	New JSP	93
3.16	Key-in Filename	94
3.17	Save All	96
3.18	Run	96
3.19	Success Message	96

3.20	ArrayList Output	97
3.21	Tomcat Before Run	97
3.22	Tomcat After Run	97
3.23	Tomcat Success Message	98
3.24	mainPage.jsp	98
3.25	headerPage.jsp Output	98
3.26	footerPage.jsp Output	99
3.27	mainPage.jsp Output	99
4.1	Creating customer.html	111
4.2	Interface of customer.html	111
4.3	Creating processCustomer.jsp	112
4.4	Example input in customer.html	114
4.5	Output after processing customer input	114
4.6	Creating currencyConversion.html	115
4.7	Interface of currencyConversion.html	116
4.8	Creating processCurrency.jsp	116
4.9	Interface of currencyConversion.html	119
4.10	Input example in currencyConversion.html	119
4.11	Processing result of currency conversion	120
4.12	Creating jspParameter.jsp	121
4.13	Creating subjectInfo.jsp	122
4.14	Output of jspParameter.jsp	123
4.15	Creating forward.jsp	124
4.16	Creating forwardInfo.jsp	125
4.17	Output of forwarded information	126
4.18	Creating insuranceQuotation.jsp	127
4.19	Insurance Quotation Interface	128
4.20	Creating processInsuranceQuo.jsp	128
4.21	Insurance Quotation Output	131
5.1	Download jstl-1.2.jar	141
7.1	Viewing the Server Log in NetBeans	189
8.1	Project Schema	213
8.2	Index	214
8.3	Employee List	214
8.4	Add Employee	214
8.5	Edit Employee	215
8.6	Delete Employee	215

Lab Module 7

SESSION MANAGEMENT, FILE UPLOAD & MVC

7.1 Objectives

By the end of this lab, students should be able to:

1. Implement a complete CRUD cycle using the strict MVC architecture (JavaBeans, DAO, Servlets, JSP).
2. Consolidate multiple operations (Login, Register, Logout, Delete Account) into a single Unified Servlet using action parameters.
3. Implement file upload functionality to store and retrieve images as BLOBs in a MySQL database.
4. Secure web applications by managing and destroying `HttpSession` objects.
5. Configure Servlet routing manually using the Deployment Descriptor (`web.xml`).
6. Trace backend execution and troubleshoot errors using the Apache Tomcat Output and Server Logs.

7.2 Introduction

In modern web applications, tracking user identity across multiple page requests is crucial. HTTP is a stateless protocol, meaning it does not remember user data between requests. To solve this, we use **Session Management**. In Java Servlets, the `HttpSession` interface allows developers to store user-specific data temporarily on the server using `setAttribute()` and retrieve them using `getAttribute()`.

Additionally, handling multimedia such as profile pictures is a standard requirement. In this lab, students will learn how to capture multipart form data (`enctype="multipart/form-data"`) using the `@MultipartConfig` annotation, store the image as binary data (BLOB) in a MySQL database, and retrieve it using Base64 encoding.

Note on Architecture: This lab enforces the **Front Controller** (Unified Servlet) design pattern within an MVC architecture. Instead of creating multiple Servlets for Registration, Login, and Logout, all requests will be routed to a single `UserServlet` via `web.xml`, which then directs traffic to the Data Access Object (DAO) based on a hidden `action` parameter.

7.3 Database Preparation

Before starting the tasks, create the necessary database and table in phpMyAdmin. Note the `MEDIUMBLOB` data type used to store the image file.

```
CREATE DATABASE lab7_db;
USE lab7_db;

CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    matric_no VARCHAR(20) UNIQUE NOT NULL,
    password VARCHAR(50) NOT NULL,
    fullname VARCHAR(100) NOT NULL,
    profile_image MEDIUMBLOB
);
```

Listing 7.1: SQL Command for Setup

7.4 Task 1: Building the Model (JavaBean & DAO)

Objective: To encapsulate data using a JavaBean and handle all database SQL logic inside a separate Data Access Object (DAO) class.

Estimated time: 30 minutes

7.4.1 Step 1: Create the StudentBean

Create a new Java Class named `StudentBean` in the `com.lab.bean` package. We will include a `base64Image` attribute to easily transfer the converted image to the View.

```
package com.lab.bean;

public class StudentBean implements java.io.Serializable {
    private String matricNo;
    private String fullname;
    private String password;
    private String base64Image;

    public StudentBean() {}

    public String getMatricNo() { return matricNo; }
    public void setMatricNo(String matricNo) { this.matricNo = matricNo; }

    public String getFullname() { return fullname; }
    public void setFullname(String fullname) { this.fullname = fullname; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public String getBase64Image() { return base64Image; }
    public void setBase64Image(String base64Image) { this.base64Image = base64Image; }
}
```

Listing 7.2: Fail: StudentBean.java

7.4.2 Step 2: Create the StudentDAO

Create a new Java Class named `StudentDAO` in the `com.lab.dao` package. This class will handle all SQL operations (register, login, and delete).

```
package com.lab.dao;

import com.lab.bean.StudentBean;
import java.io.ByteArrayOutputStream;
import java.io.InputStream;
```

```

import java.sql.Blob;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.Base64;

public class StudentDAO {

    private Connection getConnection() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/lab7_db",
            "root", "");
    }

    // --- REGISTER LOGIC ---
    public boolean registerStudent(StudentBean student, InputStream imageStream) {
        try (Connection conn = getConnection()) {
            String sql =
                "INSERT INTO students (matric_no, password, fullname, profile_image) VALUES"
                "(?, ?, ?, ?)";
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, student.getMatricNo());
            pstmt.setString(2, student.getPassword());
            pstmt.setString(3, student.getFullname());
            pstmt.setBlob(4, imageStream); // Set BLOB data

            return pstmt.executeUpdate() > 0;
        } catch (Exception e) {
            e.printStackTrace();
            return false;
        }
    }

    // --- LOGIN LOGIC ---
    public StudentBean loginStudent(String matricNo, String password) {
        StudentBean student = null;
        try (Connection conn = getConnection()) {
            String sql = "SELECT * FROM students WHERE matric_no = ? AND password = ?";
            PreparedStatement pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, matricNo);

```

```

        pstmt.setString(2, password);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            student = new StudentBean();
            student.setMatricNo(rs.getString("matric_no"));
            student.setFullname(rs.getString("fullname"));

            // Fetch BLOB and convert to Base64 String
            Blob blob = rs.getBlob("profile_image");
            if (blob != null) {
                InputStream inputStream = blob.getBinaryStream();
                ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
                byte[] buffer = new byte[4096];
                int bytesRead = -1;
                while ((bytesRead = inputStream.read(buffer)) != -1) {
                    outputStream.write(buffer, 0, bytesRead);
                }
                byte[] imageBytes = outputStream.toByteArray();
                student.setBase64Image(Base64.getEncoder().encodeToString(imageBytes));
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return student;
}

// --- DELETE LOGIC ---
public boolean deleteStudent(String matricNo) {
    try (Connection conn = getConnection()) {
        String sql = "DELETE FROM students WHERE matric_no = ?";
        PreparedStatement pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, matricNo);
        return pstmt.executeUpdate() > 0;
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
}
}

```

7.5 Task 2: The Unified Controller (UserServlet)

Objective: To consolidate multiple operations (Login, Logout, Register, Delete Account) into a single Servlet that directs traffic to the DAO based on an `action` parameter.

Estimated time: 20 minutes

7.5.1 Step 1: Create UserServlet with web.xml Mapping

1. Create a new Servlet named `UserServlet` in the `com.lab.controller` package.
2. **CRUCIAL:** In the wizard, **CHECK** the box that says **"Add information to deployment descriptor (web.xml)"**. Ensure there is no `@WebServlet` annotation generated in the class.
3. Add the `@MultipartConfig` annotation manually above the class definition to allow file uploads.

```
package com.lab.controller;

import com.lab.bean.StudentBean;
import com.lab.dao.StudentDAO;
import java.io.IOException;
import java.io.InputStream;
import javax.servlet.ServletException;
import javax.servlet.annotation.MultipartConfig;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import javax.servlet.http.Part;

@MultipartConfig(maxFileSize = 16177215) // max 16 MB required for image upload
public class UserServlet extends HttpServlet {

    private StudentDAO studentDAO = new StudentDAO();
```

```

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    response.setContentType("text/html;charset=UTF-8");

    // Read the hidden parameter to decide which action to execute
    String action = request.getParameter("action");

    if ("register".equals(action)) {
        // --- 1. HANDLE REGISTRATION ---
        StudentBean newStudent = new StudentBean();
        newStudent.setMatricNo(request.getParameter("matricNo"));
        newStudent.setFullname(request.getParameter("fullname"));
        newStudent.setPassword(request.getParameter("password"));

        InputStream inputStream = null;
        Part filePart = request.getPart("profileImage");
        if (filePart != null && filePart.getSize() > 0) {
            inputStream = filePart.getInputStream();
        }

        boolean isSuccess = studentDAO.registerStudent(newStudent, inputStream);
        if (isSuccess) {
            response.getWriter().println("Registration Successful!
            <a href='login.html'>Login here</a>");
        } else {
            response.getWriter().println("Registration Failed!");
        }

    } else if ("login".equals(action)) {
        // --- 2. HANDLE LOGIN ---
        String matric = request.getParameter("matricNo");
        String pass = request.getParameter("password");

        StudentBean student = studentDAO.loginStudent(matric, pass);

        if (student != null) {
            // Login successful: Store the entire bean in the Session
            HttpSession session = request.getSession();

```



```

        session.setAttribute("loggedUser", student);
        response.sendRedirect("dashboard.jsp");
    } else {
        response.getWriter().println("Invalid Credentials!
        <a href='login.html'>Try Again</a>");
    }
}
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String action = request.getParameter("action");
    HttpSession session = request.getSession(false);

    if ("logout".equals(action)) {
        // --- 3. HANDLE LOGOUT ---
        if (session != null) {
            session.invalidate(); // Destroy session
        }
        response.sendRedirect("login.html");
    }
    else if ("delete".equals(action)) {
        // --- 4. HANDLE DELETE ACCOUNT ---
        if (session != null && session.getAttribute("loggedUser") != null) {
            StudentBean user = (StudentBean) session.getAttribute("loggedUser");
            studentDAO.deleteStudent(user.getMatricNo());
            session.invalidate(); // Destroy session after account deletion
        }
        response.sendRedirect("register.html");
    }
}
}

```

Listing 7.4: Fail: UserServlet.java

7.6 Task 3: The Views and Session Security

Objective: To create the HTML forms containing hidden action fields and a secure

dashboard using JSP.

Estimated time: 20 minutes

7.6.1 Step 1: Registration Form

Create `register.html`. Notice the hidden input field that sends `action=register` to the `UserController`.

```
<!DOCTYPE html>
<html>
<head><title>Student Registration</title></head>
<body>
  <h2>Register New Student</h2>
  <form action="UserController" method="POST" enctype="multipart/form-data">
    <input type="hidden" name="action" value="register">

    Matric No: <input type="text" name="matricNo" required><br><br>
    Full Name: <input type="text" name="fullname" required><br><br>
    Password: <input type="password" name="password" required><br><br>
    Profile Picture: <input type="file" name="profileImage" accept="image/*" required>
    <br>
    <br>
    <input type="submit" value="Register">
  </form>
</body>
</html>
```

Listing 7.5: Fail: `register.html`

7.6.2 Step 2: Login Form

Create `login.html`. This form sends `action=login`.

```
<!DOCTYPE html>
<html>
<body>
  <h2>Student Login</h2>
  <form action="UserController" method="POST">
    <input type="hidden" name="action" value="login">

    Matric No: <input type="text" name="matricNo" required><br><br>
    Password: <input type="password" name="password" required><br><br>
```

```

        <input type="submit" value="Login">
    </form>
</body>
</html>

```

Listing 7.6: Fail: login.html

7.6.3 Step 3: Secure Dashboard JSP

Create `dashboard.jsp`. This page verifies the session. If the session exists, it retrieves the `StudentBean` object directly from the session to display the name and Base64 image. Notice the Logout and Delete links pass the `action` parameter via GET method.

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="com.lab.bean.StudentBean"%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Student Dashboard</title>

    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
    rel="stylesheet">

    <style>
        /* Custom CSS minor untuk saiz dan bingkai gambar sahaja */
        .profile-img {
            width: 160px;
            height: 160px;
            object-fit: cover;
            border: 5px solid #198754; /* Warna hijau success Bootstrap */
        }
    </style>
</head>
<body class="bg-light">

    <%
        // 1. Session Verification (Security Check)
        response.setContentType("text/html; charset=UTF-8");
        StudentBean user = (StudentBean) session.getAttribute("loggedUser");
    %>

```

```

        if(user == null) {
            response.sendRedirect("login.html");
            return;
        }
    %>

<div class="container d-flex justify-content-center align-items-center"
style="min-height: 100vh;">

    <div class="card shadow-lg border-0 rounded-4 p-4 p-md-5 text-center"
style="max-width: 600px; width: 100%;">
        <div class="card-body">

            <h2 class="fw-bold text-dark mb-2">Welcome, <%= user.getFullname() %>!</h2>
            <p class="text-muted fs-5 mb-4">Matric Number: <strong class="text-secondary">

            <div class="d-flex justify-content-center gap-3 flex-wrap">

                <a href="UserServlet?action=logout" class="btn btn-dark px-4 py-2
                fw-semibold">Logout</a>

                <a href="UserServlet?action=delete"
                    onclick="return confirm('WARNING: Are you sure you want to delete
                    your account permanently?');"
                    class="btn btn-danger px-4 py-2 fw-semibold">Delete Account</a>
            </div>

        </div>
    </div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/
bootstrap.bundle.min.js"></script>
</body>
</html>

```

Listing 7.7: Fail: dashboard.jsp

7.7 Task 4: Debugging and Viewing Server Logs

Objective: To learn how to trace and troubleshoot backend errors (e.g., HTTP 500) by inspecting the server logs and Output window in NetBeans.

Estimated time: 15 minutes

When developing web applications, you will inevitably encounter the dreaded **"HTTP Status 500 - Internal Server Error"** on your browser. The browser rarely tells you the exact line of code that caused the problem. To find the root cause, you must look at the backend server logs.

7.7.1 Step 1: Simulate a Backend Error

To learn how to debug effectively, we must first create a deliberate error and prepare a clean environment to catch it.

1. Open your `StudentDAO.java` file.
2. Find the `getConnection()` method and intentionally break the database password or URL (e.g., change `"root"` to `"wronguser"`).
3. **CRUCIAL DEBUGGING TIP:** Before you run the project, open the **Output** window at the bottom of NetBeans (or press `Ctrl + 4`), click the **Apache Tomcat** tab, right-click anywhere inside the text area, and select **Clear** (or press `Ctrl + L`).
 - *Why?* Clearing the old logs ensures that when the error happens, you are not confused by past errors. It makes it extremely easy to identify exactly which action triggered the error and at exactly what line it occurred.
4. Now, run your project and try to Login or Register.
5. Your browser will likely freeze, show a blank page, or display an HTTP 500 error. Do not panic; this is expected!

7.7.2 Step 2: Inspecting the Apache Tomcat Output Window

Whenever your Java code hits an exception and calls `e.printStackTrace();`, the details are printed directly to the server output.

1. Go back to NetBeans and look at the freshly cleared **Output** window (**Apache Tomcat** tab).

2. Because you cleared it earlier, the only red text you see is the direct result of your recent action!
3. Scroll through the error trace. You should clearly see a `java.sql.SQLException: Access denied for user 'wronguser' or something similar.`
4. Look closely at the trace lines; it will point you to the **exact line number** in your `StudentDAO.java` where the code failed.

7.7.3 Step 3: Viewing the Full Server Log via Services Tab

Sometimes, errors occur before your application even runs (e.g., a port conflict or a missing JAR file). For this, you need the full historical server log.

1. On the left side of NetBeans, locate the **Services** tab (next to Projects and Files).
2. Expand the **Servers** node.
3. Right-click on **Apache Tomcat** (or your respective server) and select **View Server Log** or **View Server Output**.

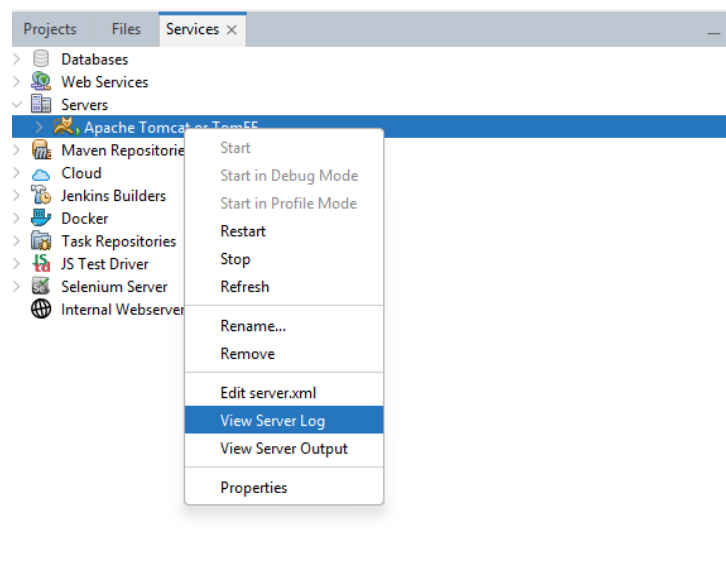


Figure 7.1: Viewing the Server Log in NetBeans

4. A new tab will open showing the detailed timestamped logs of the server. This is where system administrators look for critical application crashes.
5. **Important:** Remember to fix the intentional error in `StudentDAO.java` before proceeding to the Exercise!

Reflections

1. Why is it highly recommended to clear the Output log before testing a specific feature that might contain bugs?
2. Why is it considered a bad practice in a production environment to display the full `e.printStackTrace()` error message directly on the user's web browser?

7.8 Exercise: Student Subject Management System (CRUD)

Based on the MVC architecture and Session Management techniques learned in this lab, you are required to extend the current application by developing a **Subject Management System**. This system must allow a logged-in student to manage their academic subjects through a full CRUD cycle.

System Requirements:

1. Database Design:

- Create a table named `registered_subjects`.
- It must at least contain: `id` (PK), `matric_no` (to link with the logged-in user), and `subject_name` or `subject_code`.

2. Functionalities (Full CRUD):

- **CREATE (Daftar):** Students can register for new subjects via a form.
- **READ (View):** Students can view a list of all subjects they have registered for in a table.
- **UPDATE (Kemaskini):** Students can edit/update the details of a registered subject.
- **DELETE (Padam):** Students can remove/delete a subject from their list.

3. Technical Constraints:

- **Architecture:** Strictly follow the MVC pattern (`SubjectBean`, `SubjectDAO`, and `SubjectServlet`).
- **Unified Controller:** Use a single `SubjectServlet` (mapped via `web.xml`) to handle all actions (`add`, `view`, `update`, `delete`) using the `action` parameter.
- **Session Management:** You **MUST** retrieve the student's identity (Matric No) from the `HttpSession` to ensure they only manage their own subjects.
- **No JSTL:** Use standard Java **Scriptlets** (`<% ... %>`) and expressions (`<%= ... %>`) to display the data and handle loops in your JSP files.

Pro Tip: Project Structure for New Modules

When adding new modules (like the Subject module), it is an industry best practice to keep your Controller and Model organized in their respective Java packages, but organize your Views (HTML/JSP) into separate folders to keep the Web Pages directory clean. For example:

Source Packages

```
com.lab.bean      -> Add SubjectBean.java
com.lab.dao       -> Add SubjectDAO.java
com.lab.controller -> Add SubjectServlet.java
```

Web Pages

```
login.html
dashboard.jsp
subject (New Folder)
    registerSubject.jsp
    updateSubject.jsp
    viewSubjects.jsp
```

Assessment Rubric (20 Marks)

Criteria	Marks
MVC Structure (5 Marks): Correct separation of concerns using SubjectBean for data and SubjectDAO for SQL logic.	5
Unified Servlet & web.xml (5 Marks): Proper implementation of a single controller with action-based routing and correct web.xml mapping.	5
Full CRUD Implementation (6 Marks): Successful implementation of Add, View, Update, and Delete functions without errors.	6
Session & Scriptlet Usage (4 Marks): Correct use of HttpSession to identify the user and appropriate use of Scriptlets for data display.	4
Total	20